CSC14120 – PARALLEL PROGRAMMING

FINAL PROJECT

1. Introduction

1.1 Problem Statement

Panorama stitching combines overlapping photos into one wide image, as phone cameras do in panorama mode. The pipeline finds distinctive points in each photo, matches them, estimates the geometric transform between the photos, and warps and blends them together.

Given **two** overlapping photos of the same scene, produce one stitched panorama

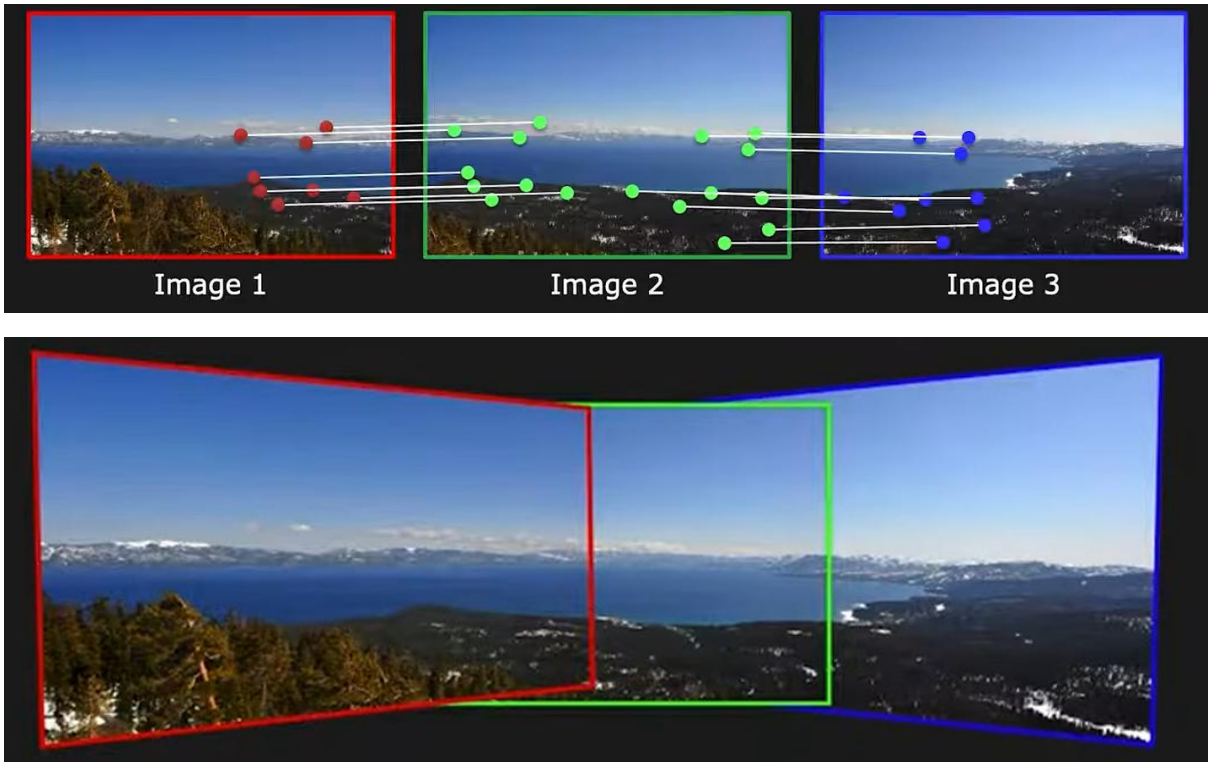


Figure 1 - Image Stitching Proble. Source: https://fpcv.cs.columbia.edu/

1.2 Dataset

Each team captures its own images at sites no other team uses, so every team works with a unique dataset

Item	Minimum	Purpose
Sites	3 different sites	Variety of scenes (e.g., one indoor, one outdoor, one of the team's choice)

Standard pairs	1 pair per site (3 pairs)	Main benchmark and results gallery
Challenging pair	1 pair	Failure analysis (e.g., low texture, repetitive patterns, moving people, night scene)

Each pair is cropped to 16:9 and resized to three sizes, so speedup can be compared across problem sizes using exactly the same scene

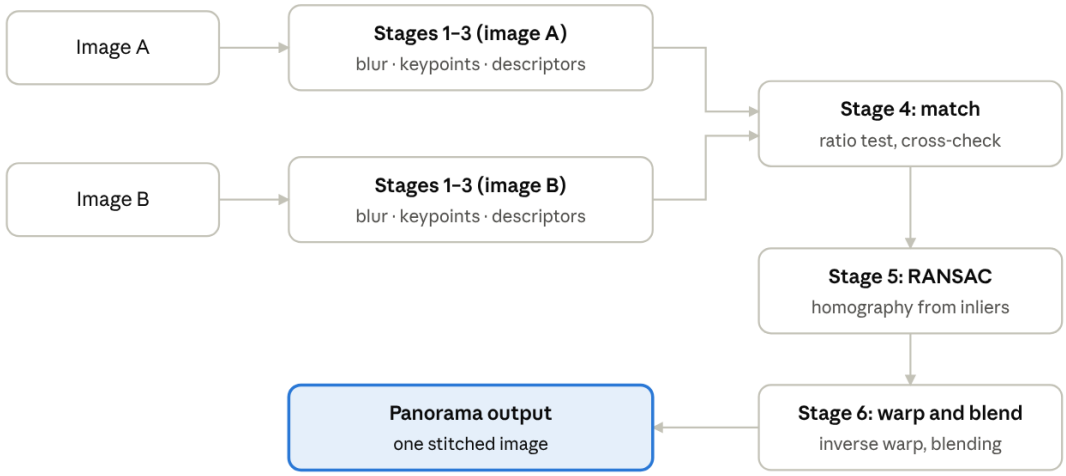
Set	Resolution	Purpose
Small	1280×720	Fast development and debugging
Medium	1920×1080	Main benchmark for the speedup table
Large	3840×2160	Shows GPU scaling; the sequential version may take several seconds here

Note when take image:

- **Rotate, don't walk.** Stand in one place and turn the camera between the two shots. Moving sideways creates parallax, which a homography cannot correct.
- **Overlap 30–50%** between the two photos.

2. Panorama Image Stitching

2.1 Pipeline



Stages 1–3 run independently on each image, so both images can be processed in parallel or in separate CUDA streams. Stages 4–6 need data from both images.

1. **Preprocessing**: convert to grayscale and smooth the image to reduce noise.
2. **Keypoint detection**: find distinctive points such as corners.
3. **Description**: compute a compact descriptor of the neighborhood around each keypoint.

4. **Matching**: for each keypoint in image A, find the most similar keypoint in image B and reject ambiguous matches.
5. **Homography estimation**: estimate the 3×3 projective transform that maps image B onto image A, robust to wrong matches.
6. **Warping and blending**: transform image B into image A's frame and blend the overlap into one output image.

2.2 Suggested methods

Teams may choose any method for each stage, if the choice is reasonable, produces a good panorama, and can be implemented efficiently. The table lists common options as a starting point, not a requirement. Teams must justify their choices in the notebook and in the oral defence, covering both output quality and parallel efficiency.

Stage	Common options	GPU notes
1. Preprocessing	Grayscale; Gaussian blur (2D or separable); image pyramid	Classic stencil. Separable filters cut work from k^2 to $2k$ per pixel; shared-memory tiles with halos reduce global reads.
2. Keypoint detection	Harris, Shi–Tomasi, FAST, DoG (SIFT-style); non-maximum suppression; top-K selection	Per-pixel response is easy to parallelize. Collecting a variable number of keypoints needs compaction (atomics or prefix sum). Top-K needs sorting or selection.
3. Description	BRIEF, ORB (rotated BRIEF), SIFT-like histograms	Binary descriptors (256 bits) are cheap to compute and compare. SIFT-like descriptors are more robust but heavier and use float histograms.
4. Matching	Brute-force nearest neighbor (Hamming or L2); Lowe's ratio test; cross-check	An all-pairs problem similar to matrix multiplication. Tiling descriptors in shared memory and `__popc` for Hamming distance help a lot.
5. Homography	RANSAC with 4-point DLT; optional refinement on inliers; variants such as PROSAC	Hypotheses are independent, so each thread, warp, or block can test one. Inlier counting is a reduction.
6. Warping and blending	Inverse warping with nearest or bilinear sampling; overwrite,	A gather per output pixel. Texture memory gives hardware bilinear interpolation. Multi-

Stage	Common options	GPU notes
	feathering, multi-band blending	band blending adds pyramid stages.

Note: Choices affect both quality and speed. For example, ORB with Hamming matching is fast and fits the GPU well, while SIFT-like features are more robust to scale changes but harder to optimize. A method that is fast but produces a badly aligned panorama will lose marks, and so will a high-quality method that the team cannot explain.

2.3 Recommended Reading Materials

- Nayar, "Image Stitching" monograph FPCV-2-4 (2025): written notes for the same module, on the Monographs page.
- R. Szeliski, *Computer Vision: Algorithms and Applications*, 2nd ed., Springer 2022, free draft at szeliski.org/Book: chapters on feature detection and matching, image alignment, and stitching.

3. Implementation Guideline

Every team submits at least 4 versions of the same pipeline, using the same methods and parameters in every version so that timing comparisons are fair.

Version	Name	Requirement
V0	Sequential	Single-thread C/C++ implementation of the team's chosen methods. This is the reference for correctness and speedup.
V1	Basic CUDA	Every stage runs on the GPU with straightforward kernels (e.g., one thread per pixel, keypoint, or match). Global memory is fine.
V2, V3, V4, …	Optimized	At least 2 optimized versions. Each version introduces one main optimization idea so its effect can be measured separately.

Rules for optimized versions

- Each version builds on the previous one, keeping earlier optimizations unless the notebook explains why one was removed.
- Each optimization must be motivated by a measured bottleneck (profiling or per-stage timing), not chosen at random.

- A version that is slower than the previous one is acceptable if the team explains why. Honest analysis of a failed idea earns marks.
- Every version must produce results consistent with V0.

Optimization menu (suggestions, not requirements)

- **Memory**: shared-memory tiling with halos, constant memory for filter weights and sampling patterns, texture memory for warping, coalesced data layouts (structure of arrays).
- **Computation**: separable convolution, `__popc` for Hamming distance, fast math intrinsics, loop unrolling, more work per thread.
- **Parallel patterns**: warp-shuffle reductions, prefix-sum compaction instead of global atomics, parallel top-K selection, one warp per RANSAC hypothesis.
- **Pipeline level**: kernel fusion (e.g., grayscale + blur), keeping all data on the GPU between stages, pinned memory, CUDA streams to process both images concurrently.
- **Algorithm level**: early termination in matching or RANSAC, image pyramids, reduced precision where quality allows.

4. Running and Benchmarking

4.1 Input and output specification

One program handles all versions, selected by a command-line flag, so every version runs on the same input.

Command line

```
./stitch <left_image> <right_image> <output_image> --version <0|1|2|...> [--runs N]
```

- Input: two RGB images. Image B is warped onto image A's frame.
- Output: the panorama image
- `--runs N`: repeat the pipeline N times and report the average time (default 5).

4.2 Performance measurement rules

All timings in the notebook must follow the same method, otherwise speedups between versions are meaningless.

- **GPU timing:** use CUDA events (`cudaEventRecord`, `cudaEventElapsedTime`) around each stage. Do not use CPU timers around asynchronous kernel launches without synchronizing.
- **CPU timing:** use `std::chrono::high_resolution_clock` for V0.
- **Warm-up:** run the pipeline once before timing, to exclude CUDA context creation and first-launch overhead.
- **Repetition:** report the average of at least 5 timed runs; also report the minimum if the variance is large.
- **What to report:** time per stage, total kernel time, host–device transfer time, and end-to-end time. Image loading and saving from disk are excluded.
- **Speedup:** report speedup against V0 and against the previous version, for every resolution.
- **Same GPU:** all versions in one comparison table must be measured on the same GPU in the same Colab session. Print the GPU name with every result.
- **Profiling evidence:** each optimized version must include profiling output supporting its motivation, e.g., an Nsight Systems timeline showing the dominant stage, or Nsight Compute metrics such as memory throughput, occupancy, or warp stall reasons for the key kernel. Screenshots or text summaries are both acceptable.

5. Project Deliverable

5.1 Submission Requirements

You must submit the following items:

1. Project Report (see next section for detail structure)

- **Jupyter Notebook (.ipynb)** as specified in Report Requirements
- Must be executable on Google Colab
- Export to PDF for archival purposes

2. Source Code Package

- **All source code files** (.cpp, .cu, .h, header files, etc.)
- **README.md** with:

- Setup instructions (dependencies, CUDA version, libraries)
- Compilation commands
- Execution instructions with example commands
- Hardware requirements (GPU model, memory)
- Expected outputs

3. Presentation Slides

4. Presentation Video (20-30 minutes)

- **Upload to YouTube** with **Unlisted** visibility
- **Include link** in your notebook report and README file
- **DO NOT upload video to Moodle** (file size restrictions)

Video Content Should Cover:

- Problem overview and approach (4-6 min)
- Live demonstration of running code (6-9 min)
- Performance results and analysis (6-9 min)
- Key optimizations and lessons learned (4-6 min)

5.2 Notebook report structure

Hard requirements

- The notebook runs from top to bottom with Runtime → Run all on a fresh Colab GPU session, without manual steps other than uploading the dataset if needed.
- All output cells are saved in the submitted file, so the report can be read without re-running it.
- Every number in the text comes from an output cell in the notebook.

Required structure.

1. **Team information:** member names, student IDs, and a contribution table showing who did what (by stage and by version).
2. **Introduction:** the problem in the team's own words and the pipeline overview.

3. **Chosen methods:** the method for each stage and why, covering quality and parallel efficiency; parameters used (e.g., number of keypoints, ratio-test threshold, RANSAC iterations).
4. **V0 Sequential:** design, code, output panorama, per-stage timing, and which stages dominate.
5. **V1 Basic CUDA:** how each stage is mapped to threads and blocks, code, verification against V0, timing and speedup.
6. **V2, V3, … Optimized versions,** each with the same five parts:
    a. Bottleneck: profiling or timing evidence showing what limits the previous version.
    b. Idea: the optimization and why it should help.
    c. Implementation: the key code changes.
    d. Result: verification against V0, timing, speedup against V0 and the previous version.
    e. Analysis: whether the speedup matched expectations and why.
7. **Final comparison:** speedup table and charts across all versions and resolutions, and the per-stage time breakdown of the best version.
8. **Results gallery:** panoramas from the provided and the team's own image pairs, including at least one failure case with an explanation.
9. **Conclusion:** lessons learned, limitations, and what the team would try next.
10. **References:** papers, tutorials, and code consulted, plus the AI tool usage declaration.

6. Academic integrity

Teams may learn from any source, but everything submitted must be understood and explained by every member; the oral defense is the main check.

- **External code and ideas:** reading papers, tutorials, and open-source code is allowed. Any code or idea taken from outside the team must be cited in the notebook at the place it is used and in the References section. Uncited copied code is treated as plagiarism.

- **Sharing between teams:** discussing ideas between teams is allowed; sharing code is not.
- **AI tools:** tools such as ChatGPT, Claude, or Copilot may be used for learning, debugging, and explanations. The notebook must include a short declaration of which tools were used and for what. Code produced with AI tools counts as the team's own only if the team can explain and modify it in the oral defense.

Honest results: all timings and images must come from the submitted code. Fabricated or edited results lead to a zero for the project.

------ END ------